



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema: Grafos
Unidad de Organización Curricular: BÁSICA
Nivel y Paralelo: Tercero- B
Alumno: Medina Chico Darlyn Monserrath
Asignatura: ESTRUCTURA DE DATOS
Docente: Ing. Jose Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en la programación con grafos.

Específicos:

- Representar ubicaciones del campus mediante nodos dentro de un grafo.
- Establecer conexiones entre ubicaciones utilizando aristas ponderadas.
- Implementar el algoritmo BFS para encontrar rutas con menor número de paradas.
- Implementar el algoritmo Dijkstra para determinar rutas de menor distancia.
- Comparar los resultados obtenidos por ambos algoritmos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Apuntes de clase
- Bibliografía virtual

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Implementar el caso de estudio del Mapa del campus y búsqueda de rutas.



INFORME

I. Introducción

Los grafos son estructuras de datos utilizadas para representar relaciones entre diferentes elementos mediante nodos y conexiones. Su aplicación es amplia en sistemas de navegación, redes sociales, telecomunicaciones y planificación de rutas.

En la presente práctica se desarrolló una aplicación en Java que modela un mapa simplificado del Campus Huachi de la Universidad Técnica de Ambato utilizando grafos representados mediante listas de adyacencia. Además, se implementaron los algoritmos BFS y Dijkstra para comparar diferentes criterios de búsqueda de rutas.

II. Marco Teórico

Grafos: Un grafo es una estructura de datos compuesta por un conjunto de vértices (nodos) y aristas (conexiones) que permiten modelar relaciones entre elementos.

Lista de Adyacencia: La lista de adyacencia es una representación eficiente de grafos donde cada nodo almacena una lista de sus vecinos conectados.

Algoritmo BFS: First Search (BFS) es un algoritmo de recorrido que explora los nodos por niveles. Permite encontrar la ruta con menor cantidad de pasos o paradas entre dos nodos.

Algoritmo de Dijkstra: El algoritmo de Dijkstra calcula el camino más corto considerando los pesos asociados a las aristas, permitiendo encontrar la ruta de menor costo o distancia.

III. Descripción del problema

Se modela el campus mediante un grafo no dirigido con pesos (distancias en metros). Los nodos representan ubicaciones y las aristas los caminos.

Nodos creados:

ID	Nombre
uta	Universidad
fisei	FISEI
idiomas	Idiomas
biblioteca	Biblioteca
estadio	Estadio
comedor	Comedor

Aristas (origen → destino, peso):

- uta → fisei (50)
- fisei → idiomas (40)
- idiomas → biblioteca (30)
- biblioteca → estadio (70)
- uta → comedor (20)
- comedor → estadio (200)

IV. Implementación

Se implementó una aplicación en Java utilizando programación orientada a objetos. Inicialmente se definieron las clases Nodo y Arista para representar las ubicaciones y conexiones del campus.



Posteriormente se creó la clase Grafo, encargada de almacenar los nodos y la lista de adyacencia. Se implementó el método agregarNodo() para registrar las diferentes ubicaciones del campus y el método agregarArista() para establecer las conexiones entre ellas junto con sus respectivas distancias.

Para el análisis de rutas se implementaron dos algoritmos:

- BFS, encargado de encontrar la ruta con menor número de paradas.
- Dijkstra, encargado de encontrar la ruta con menor distancia acumulada.

I. Métodos implementados

Creación de nodos:

```
grafo.agregarNodo(id: "uta", nombre: "Universidad");  
grafo.agregarNodo(id: "fisei", nombre: "FISEI");  
grafo.agregarNodo(id: "idiomas", nombre: "Idiomas");  
grafo.agregarNodo(id: "biblioteca", nombre: "Biblioteca");  
grafo.agregarNodo(id: "estadio", nombre: "Estadio");  
grafo.agregarNodo(id: "comedor", nombre: "Comedor");
```

Creación de aristas:

```
grafo.agregarArista(origen: "uta", destino: "fisei", peso: 50);  
grafo.agregarArista(origen: "fisei", destino: "idiomas", peso: 40);  
grafo.agregarArista(origen: "idiomas", destino: "biblioteca", peso: 30);  
grafo.agregarArista(origen: "biblioteca", destino: "estadio", peso: 70);
```

BFS:



```
public List<String> bfs(String inicio, String fin) {
    Queue<List<String>> cola = new LinkedList<>();
    Set<String> visitados = new HashSet<>();
    List<String> caminoInicial = new ArrayList<>();
    caminoInicial.add(inicio);
    cola.add(caminoInicial);
    visitados.add(inicio);
    while (!cola.isEmpty()) {
        List<String> camino = cola.poll();
        String actual =
            camino.get(camino.size() - 1);
        if (actual.equals(fin)) {
            return camino;
        }
        for (Arista arista : adyacencia.get(actual)) {
            if (!visitados.contains(arista.destino)) {
                visitados.add(arista.destino);
                List<String> nuevoCamino =
                    new ArrayList<>(camino);
                nuevoCamino.add(arista.destino);
                cola.add(nuevoCamino);
            }
        }
    }
    return null;
}
```

Dijkstra:



```
public List<String> dijkstra(String inicio, String fin) {
    Map<String, Integer> distancias =
        new HashMap<>();
    Map<String, String> anteriores =
        new HashMap<>();
    PriorityQueue<String> cola =
        new PriorityQueue<>(
            Comparator.comparingInt(
                distancias::get
            )
        );
    // Inicializar distancias
    for (String nodo : nodos.keySet()) {
        distancias.put(nodo, Integer.MAX_VALUE);
    }
    distancias.put(inicio, value: 0);
    cola.add(inicio);
    while (!cola.isEmpty()) {
        String actual = cola.poll();
        for (Arista arista : adyacencia.get(actual)) {
            int nuevaDistancia =
                distancias.get(actual)
                + arista.peso;
            if (nuevaDistancia <
                distancias.get(arista.destino)) {
                distancias.put(
                    arista.destino,
                    nuevaDistancia
                );
                anteriores.put(
                    arista.destino,
                    actual
                );
                cola.add(arista.destino);
            }
        }
    }

    // Reconstruir camino
    List<String> camino = new ArrayList<>();
    String actual = fin;
    while (actual != null) {
        camino.add(index: 0, actual);
        actual = anteriores.get(actual);
    }
    return camino;
}
```

II. Resultados obtenidos

```
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\Mapa de grafos\Ap04-grafos-completos & "C:\Program Files\Micr
osoft\Adoptium\jdk-21.0.11-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\Usuario\AppData\Local\Programs\Code\user\workspace\storage\
Software\code\paraf210123ad092226\redhot_java\jdk_21\se\Ap04-grafos-completos\lib\out\bin" "Ap04-grafos"
===== BFS =====
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)

===== DIKSTRA =====
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\Mapa de grafos\Ap04-grafos-completos
```

III. Análisis y comparación



Algoritmo	Ruta	Distancia total	# Paradas
BFS	uta → fisei → idiomas → biblioteca → estadio	$50+40+30+70 = 190\text{m}$	5 nodos
Dijkstra	uta → comedor → estadio	$20+200 = 220\text{m}$	3 nodos

Los resultados obtenidos muestran que BFS y Dijkstra generan rutas diferentes debido a sus criterios de búsqueda.

El algoritmo BFS seleccionó la ruta Universidad → Comedor → Estadio, ya que busca el camino con menor número de paradas o saltos entre nodos.

Por otro lado, el algoritmo Dijkstra seleccionó la ruta Universidad → FISEI → Idiomas → Biblioteca → Estadio, debido a que busca minimizar la distancia total recorrida considerando los pesos de las aristas.

Aunque la ruta encontrada por BFS tiene menos paradas, su distancia total es de 220 metros (20 + 200). En cambio, la ruta obtenida por Dijkstra tiene una distancia total de 190 metros (50 + 40 + 30 + 70), por lo que resulta más eficiente en términos de recorrido.

Esta comparación permite comprender que la ruta con menos paradas no siempre corresponde a la ruta más corta en distancia.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de la vida real, en este caso calcular las rutas entre dos ubicaciones., utilizando grafos

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Los grafos constituyen la estructura de datos más completa por su flexibilidad y capacidad de abstracción de una realidad objetiva.

2.10 Recomendaciones

Tener en cuenta las condiciones específicas de cada situación para decidir qué estructura de datos utilizar.

2.11 GitHub

<https://github.com/Monse1806/Estructura-de-datos.git>